
SEMANTIC SEGMENTATION FOR LANDSLIDE MAPPING *USING A U-NET ARCHITECTURE* FROM SATELLITE IMAGERY

COMPUTER VISION FINAL PROJECT

Name : Naufal Bagas Ganefriady

NIM : 23/516913/PA/22138

Initiation and Background

Why It Matters

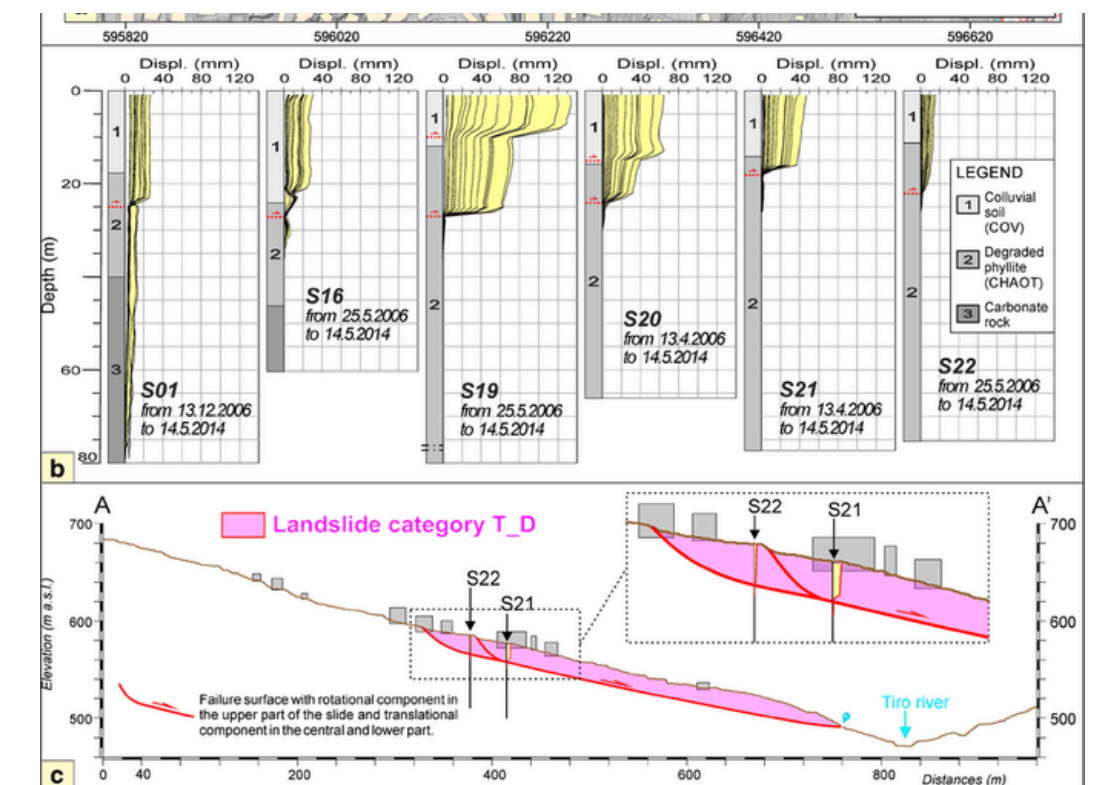
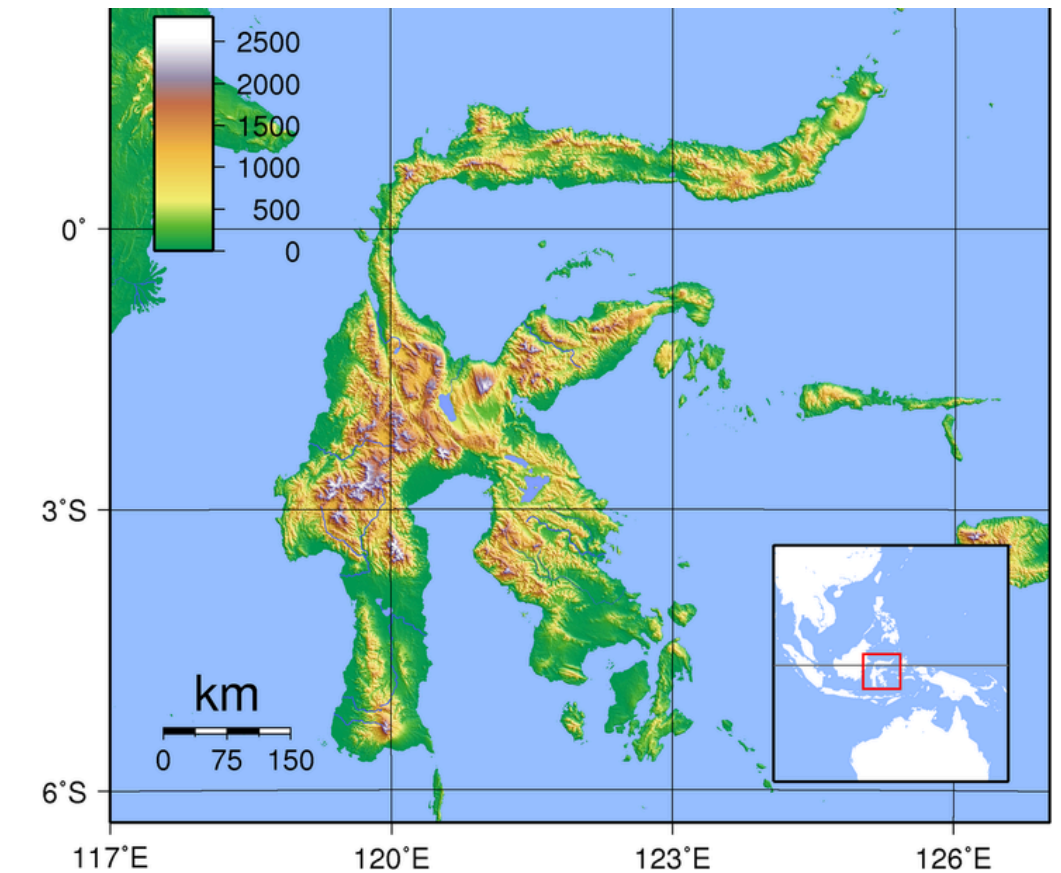
Landslides, especially in Indonesia South Sulawesi's mining regions (nickel) are very often affected by these disasters, which can cause damage, due to the **failure for carry out very important geotechnical assessment monitoring**.

Challenges

Manual detection is necessary to use a lot of manpower and resources, increasing in cost or effort, also its accuracy depends on many factors that are calculated from various thresholds. This can also cause a lot of material losses to the local community.

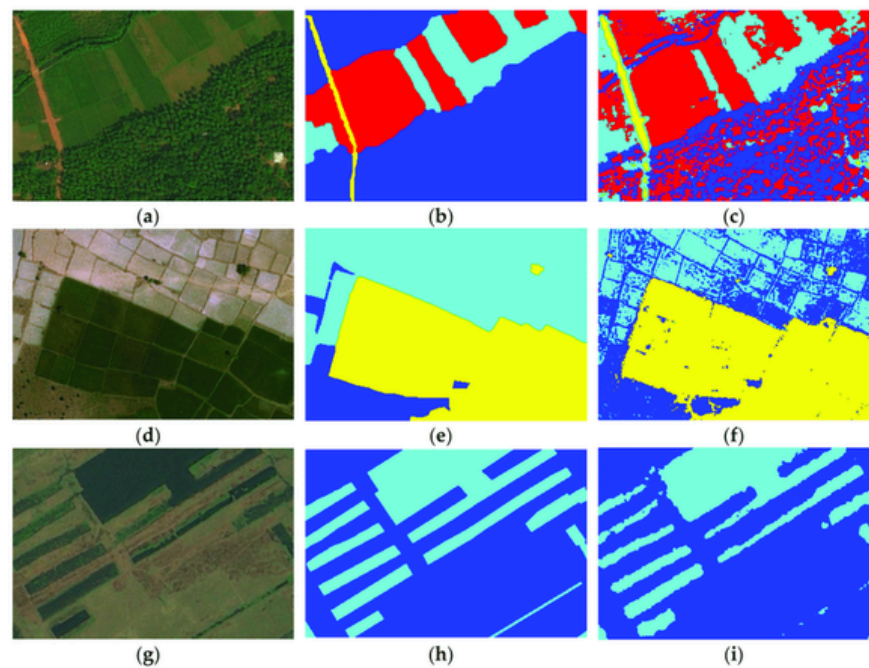
Solution

Computer Vision task, using Semantic Segmentation and U-Net Architecture, will gives precise mapping of landslide areas from satellite imagery.

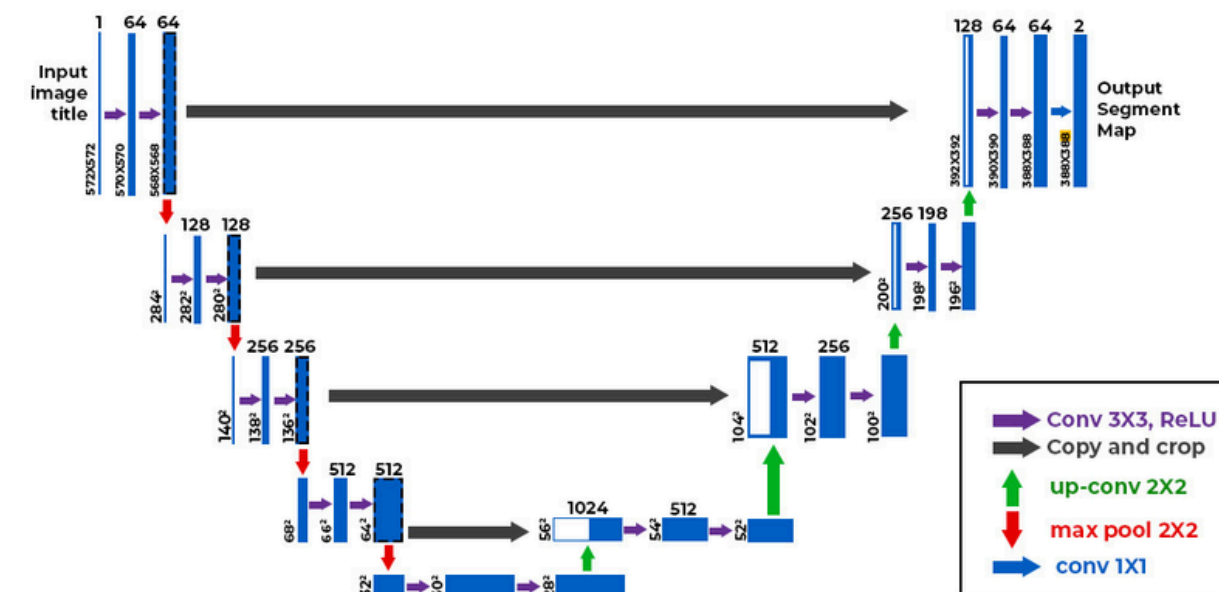


FUNDAMENTAL CONCEPT

Semantic Why U-Net?



Semantic segmentation is chosen for this case because it involves analyzing an image, identifying key features, and labeling each pixel

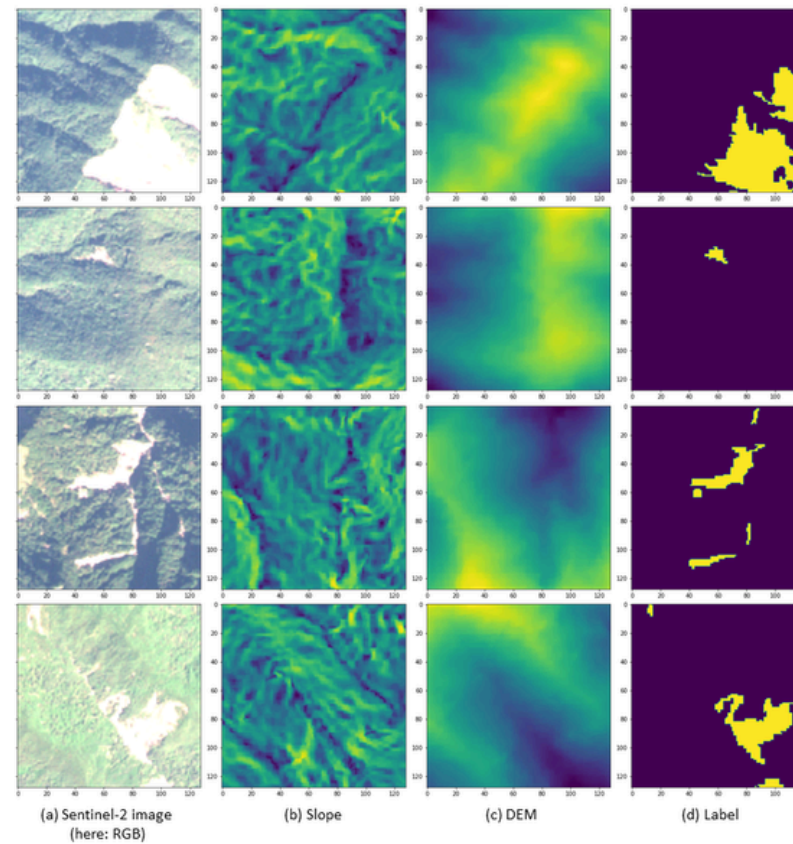


U-Net was chosen because of its unique U-Set structure, with narrowing and widening paths, excelling in precise landslide area segmentation from satellite imagery by combining detailed features and context,

How U-Net works:

1. **Start:** Take a small image
2. **Reduce & Analyze:** The image is narrowed down (like zooming out) to find key features.
3. **Zoom in & Detail:** The image is widened again (like zooming in) while adding more detail from the first step.
4. **Result:** Get a map that clearly shows the landslide area

DATASET



Version 1 (8.97 GB)

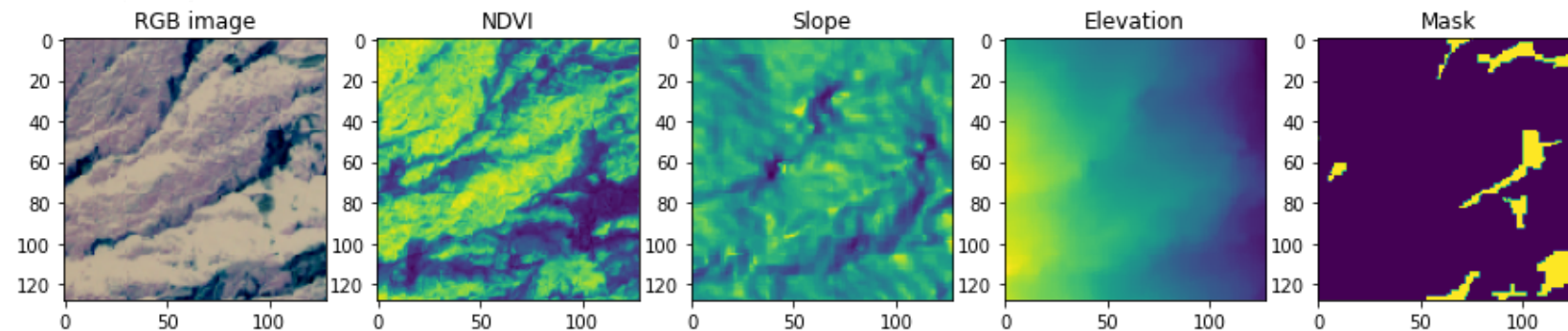
- TestData
 - img
- TrainData
 - img
 - mask
- ValidData
 - img

Dataset used in this project is from Landslide4Sense (Open-source) which consisting of **3799, 245, and 800 image patches**. Each image is a composite of 14 bands that include:

- Multispectral data from Sentinel-2
- Slope data from ALOS PALSAR
- Digital elevation model (DEM) from ALOS PALSAR

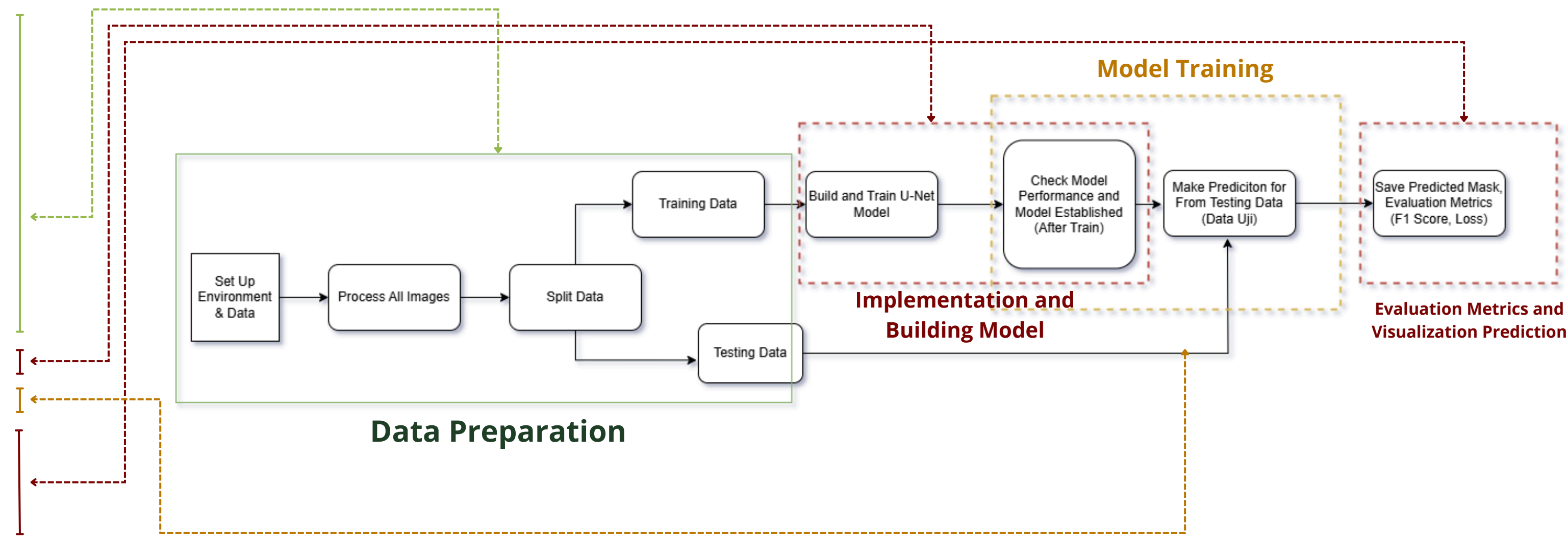
My data source includes multi-channel satellite imagery (RGB, NDVI, DEM, Slope) **provided** by the dataset, with each channel crucial for landslide detection as the **dataset is made for this purpose**

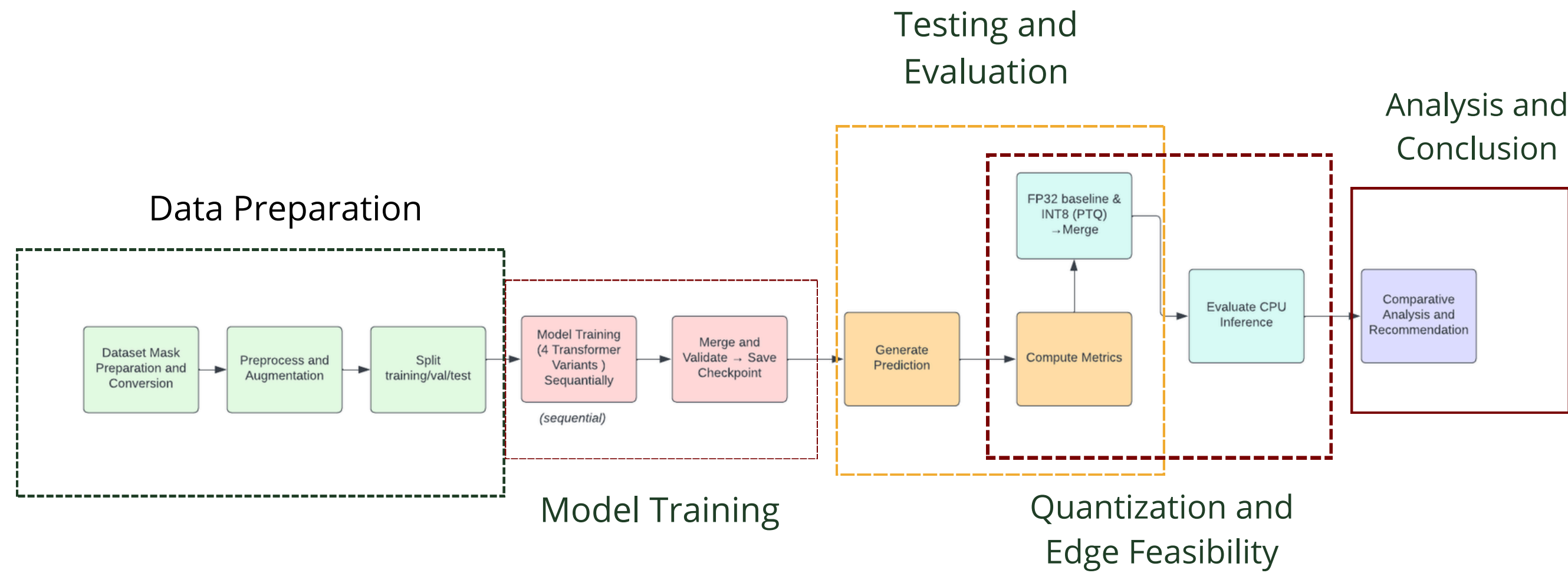
- RGB: Captures natural colors
- NDVI: Shows vegetation health
- DEM: Maps elevation
- Slope: Highlights land steepness



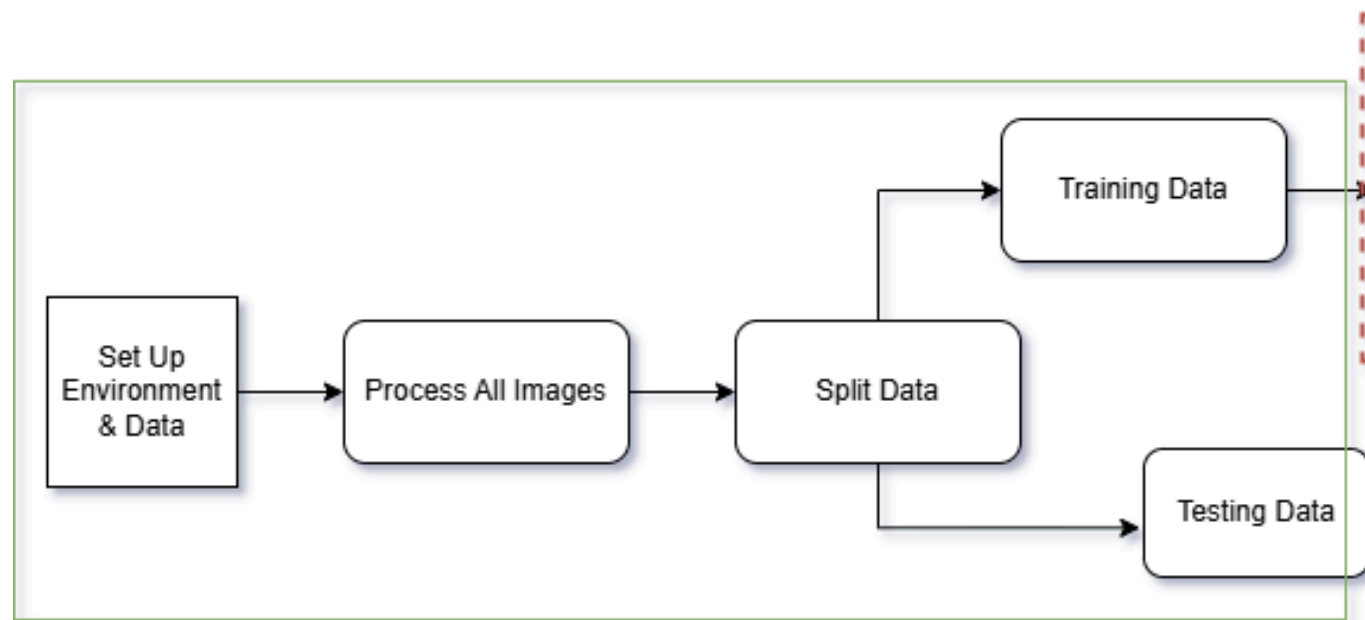
MODEL FLOWCHART

- Installation of required libraries and basic setup
- Testing the dataset
- Using all dataset
 - Train with RGB, NDVI, DEM, and Slope
 - Testing min, max values in train data
 - Custom loss function (Dice Loss)
 - Visualization of the training data**
 - Validation split
- Unet Model
- Prediction
- Validation data
 - Prediction for validation data
 - Visualization of validation image





DATA PREPARATION



Setup Environment and Data : Prepares the work environment by installing libraries (TensorFlow, Matplotlib), importing them, and connecting Google Colab to Google Drive for data access.

Process all images: Checks if image data can be accessed and read, displaying a sample image and its mask.

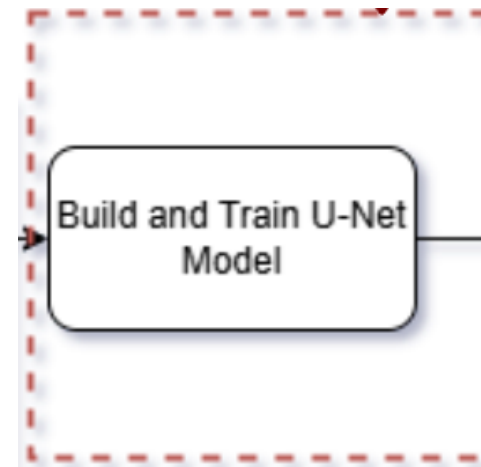
Dataset & Model Preparation:

- Image Data Combination: Loads all training images and masks, normalizes RGB, NDVI, Slope, and Elevation
- Data Check: Verifies min/max values to proper normalization
- Custom Loss Function: Defines Dice Loss to measure model prediction accuracy against actual masks. lower values indicate better segmentation.
- **Data Split:** Divides data into 80% training and 20% validation sets, clears unused variables to save memory.

Testing the dataset

```
# Testing the dataset
path_single = r"TrainData/img/image_2000.h5"
path_single_mask = r"TrainData/mask/mask_2000.h5"
```

```
[8] f_data = np.zeros((1, 128, 128, 3))
with h5py.File(path_single) as hdf:
    ls = list(hdf.keys())
    print("ls", ls)
    data = np.array(hdf.get('img'))
    print("input data shape:", data.shape)
    plt.imshow(data[:, :, 3:0:-1])
```



BUILDING U-NET MODEL AND TRAINING

Contraction Path (c1 to c5)

- Makes the image smaller step by step using special layers that find important patterns.
- Uses more detailed checks (from 16 to 256) while keeping the image size smaller.

Expansion Path (u6 to u9)

- Makes the image bigger again using layers that add back details from the shrinking part (c4 to c1).
- Reduces the detailed checks back to 16, finishing with a simple layer to show the final map.

Model Compilation and Training

- Set up with a learning method (Adam), a way to measure mistakes (binary cross-entropy), and checks (accuracy, F1, precision, recall).
- Learns by adjusting itself using example data to get better at spotting landslides.
- Use 100 epoch and callbacks for saving best model

```
from utils import recall_m, precision_m, f1_m
def unet_model(IMG_WIDTH, IMG_HEIGHT, IMG_CHANNELS):
    inputs = tf.keras.layers.Input((IMG_WIDTH, IMG_HEIGHT, IMG_CHANNELS))

    # Converted inputs to floating
    #s = tf.keras.layers.Lambda(lambda x: x / 255)(inputs)

    #Contraction path
    c1 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(inputs)
    c1 = tf.keras.layers.Dropout(0.1)(c1)
    c1 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c1)
    p1 = tf.keras.layers.MaxPooling2D((2, 2))(c1)

    c2 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p1)
    c2 = tf.keras.layers.Dropout(0.1)(c2)
    c2 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c2)
    p2 = tf.keras.layers.MaxPooling2D((2, 2))(c2)

    c3 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p2)
    c3 = tf.keras.layers.Dropout(0.2)(c3)
    c3 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c3)
    p3 = tf.keras.layers.MaxPooling2D((2, 2))(c3)

    c4 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p3)
    c4 = tf.keras.layers.Dropout(0.2)(c4)
    c4 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c4)
    p4 = tf.keras.layers.MaxPooling2D(pool_size=(2, 2))(c4)

    c5 = tf.keras.layers.Conv2D(256, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(p4)
    c5 = tf.keras.layers.Dropout(0.3)(c5)
    c5 = tf.keras.layers.Conv2D(256, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c5)

    #Expansive path
    u6 = tf.keras.layers.Conv2DTranspose(128, (2, 2), strides=(2, 2), padding='same')(c5)
    u6 = tf.keras.layers.concatenate([u6, c4])
    c6 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u6)
    c6 = tf.keras.layers.Dropout(0.2)(c6)
    c6 = tf.keras.layers.Conv2D(128, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c6)

    u7 = tf.keras.layers.Conv2DTranspose(64, (2, 2), strides=(2, 2), padding='same')(c6)
    u7 = tf.keras.layers.concatenate([u7, c3])
    c7 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u7)
    c7 = tf.keras.layers.Dropout(0.2)(c7)
    c7 = tf.keras.layers.Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c7)

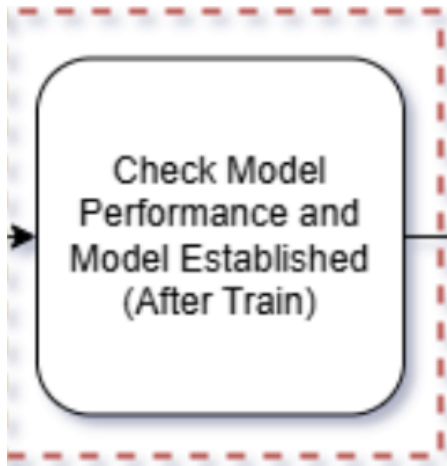
    u8 = tf.keras.layers.Conv2DTranspose(32, (2, 2), strides=(2, 2), padding='same')(c7)
    u8 = tf.keras.layers.concatenate([u8, c2])
    c8 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u8)
    c8 = tf.keras.layers.Dropout(0.1)(c8)
    c8 = tf.keras.layers.Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c8)

    u9 = tf.keras.layers.Conv2DTranspose(16, (2, 2), strides=(2, 2), padding='same')(c8)
    u9 = tf.keras.layers.concatenate([u9, c1], axis=3)
    c9 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(u9)
    c9 = tf.keras.layers.Dropout(0.1)(c9)
    c9 = tf.keras.layers.Conv2D(16, (3, 3), activation='relu', kernel_initializer='he_normal', padding='same')(c9)

    outputs = tf.keras.layers.Conv2D(1, (1, 1), activation='sigmoid')(c9)

    model = tf.keras.Model(inputs=[inputs], outputs=[outputs])
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy', f1_m, precision_m, recall_m])

    return model
```

TRAINING PROGRESS MODEL

```
model = unet_model(128, 128, 6)
# model.summary()
checkpointer = tf.keras.callbacks.ModelCheckpoint("best_model.h5", monitor="val_f1_m", verbose=1, save_best_only=True, mode="max")
# earlyStopping = tf.keras.callbacks.EarlyStopping(monitor='val_f1_m', patience=10, verbose=1, mode='max')

callbacks = [
    # earlyStopping,
    checkpointer
]
history = model.fit(x_train, y_train, batch_size=16,
                    epochs=100,
                    verbose = 2,
                    validation_data=(x_valid, y_valid),
                    callbacks=callbacks)

model.save("model_save.h5")
```

190/190 - 8s - 42ms/step - accuracy: 0.9917 - f1_m: 0.7982 - loss: 0.0199 - precision_m: 0.8309 - recall_m: 0.7694 - val_accuracy: 0.71257
Epoch 76/100

Epoch 76: val_f1_m did not improve from 0.71257
190/190 - 10s - 54ms/step - accuracy: 0.9917 - f1_m: 0.8005 - loss: 0.0200 - precision_m: 0.8348 - recall_m: 0.7716 - val_accuracy: 0.71257
Epoch 77/100

Epoch 77: val_f1_m did not improve from 0.71257
190/190 - 10s - 53ms/step - accuracy: 0.9917 - f1_m: 0.8021 - loss: 0.0199 - precision_m: 0.8376 - recall_m: 0.7712 - val_accuracy: 0.71257
Epoch 78/100

Epoch 78: val_f1_m did not improve from 0.71257
190/190 - 10s - 53ms/step - accuracy: 0.9916 - f1_m: 0.7963 - loss: 0.0204 - precision_m: 0.8343 - recall_m: 0.7649 - val_accuracy: 0.71257
Epoch 79/100

Epoch 79: val_f1_m did not improve from 0.71257
190/190 - 8s - 43ms/step - accuracy: 0.9917 - f1_m: 0.7988 - loss: 0.0201 - precision_m: 0.8344 - recall_m: 0.7687 - val_accuracy: 0.71257
Epoch 80/100

Epoch 80: val_f1_m did not improve from 0.71257
190/190 - 10s - 54ms/step - accuracy: 0.9918 - f1_m: 0.7989 - loss: 0.0199 - precision_m: 0.8366 - recall_m: 0.7669 - val_accuracy: 0.71257
Epoch 81/100

Epoch 81: val_f1_m did not improve from 0.71257
190/190 - 10s - 53ms/step - accuracy: 0.9910 - f1_m: 0.7782 - loss: 0.0224 - precision_m: 0.8229 - recall_m: 0.7427 - val_accuracy: 0.71257
Epoch 82/100

Epoch 82: val_f1_m did not improve from 0.71257
190/190 - 8s - 42ms/step - accuracy: 0.9917 - f1_m: 0.7985 - loss: 0.0200 - precision_m: 0.8355 - recall_m: 0.7670 - val_accuracy: 0.71257
Epoch 83/100

Epoch 83: val_f1_m did not improve from 0.71257
190/190 - 10s - 55ms/step - accuracy: 0.9919 - f1_m: 0.8059 - loss: 0.0196 - precision_m: 0.8391 - recall_m: 0.7770 - val_accuracy: 0.71257
Epoch 84/100

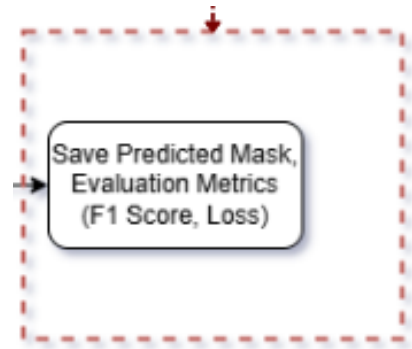
This section shows the Training Progress Log, detailing the U-Net model's learning over time. It displays key metrics per epoch (training cycle), such as:

- Loss: How well the model is doing (lower is better).
- Accuracy: Overall correct predictions.
- F1-Score, Precision, Recall: Crucial metrics for landslide detection, showing balance and correctness.
- "val_" metrics: Performance on unseen validation data, vital for detecting overfitting.

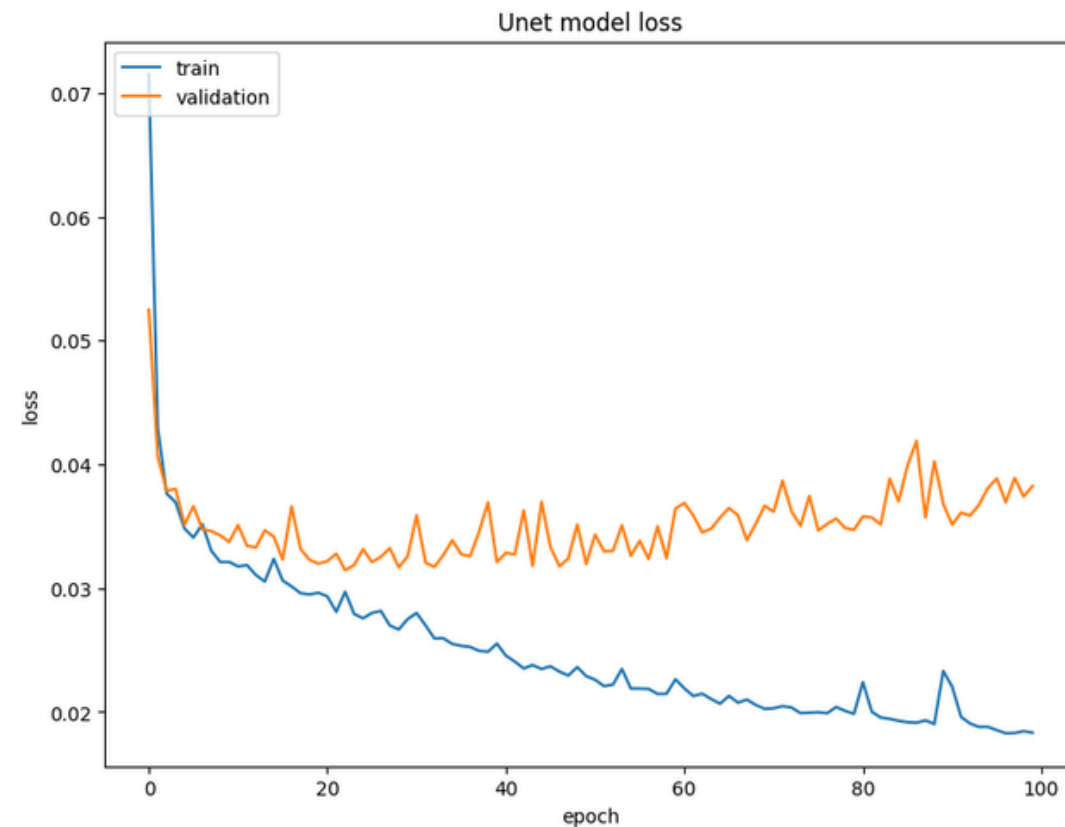
This log helps **verify** if the model is learning effectively and generalizing well. The Next Section will go into the visualization, which can be determine as evaluation metrix too.

EVALUATION METRICS

(PLOTting FROM TRAINING PROGRESS)

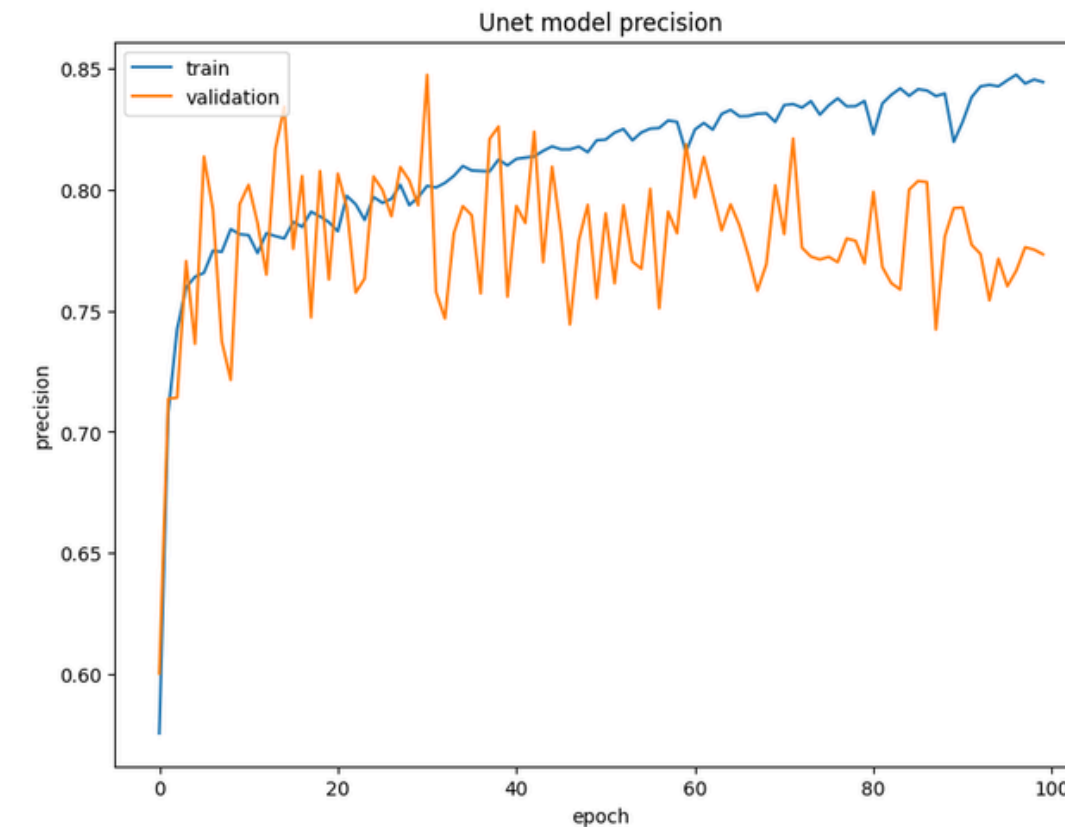


Evaluation Metrics and
Visualization Prediction



- **Loss:**

- Pro: Training loss drops well, showing the model effectively learns landslide patterns. Validation loss quickly stabilizes, indicating it finds a good initial fit for unseen landslide data.
- Con: The increasing gap signals overfitting, meaning the model is too specific to the trained landslide examples, not generalizing perfectly to new ones.



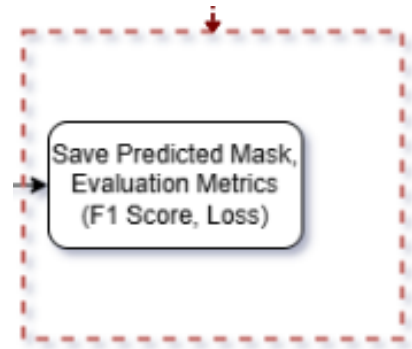
- **Precision:**

- Pro: High precision means when the model says "this is a landslide," it's very likely correct, minimizing false alarms.
- Con: Validation precision fluctuates, showing inconsistent reliability in avoiding false alarms on new areas.

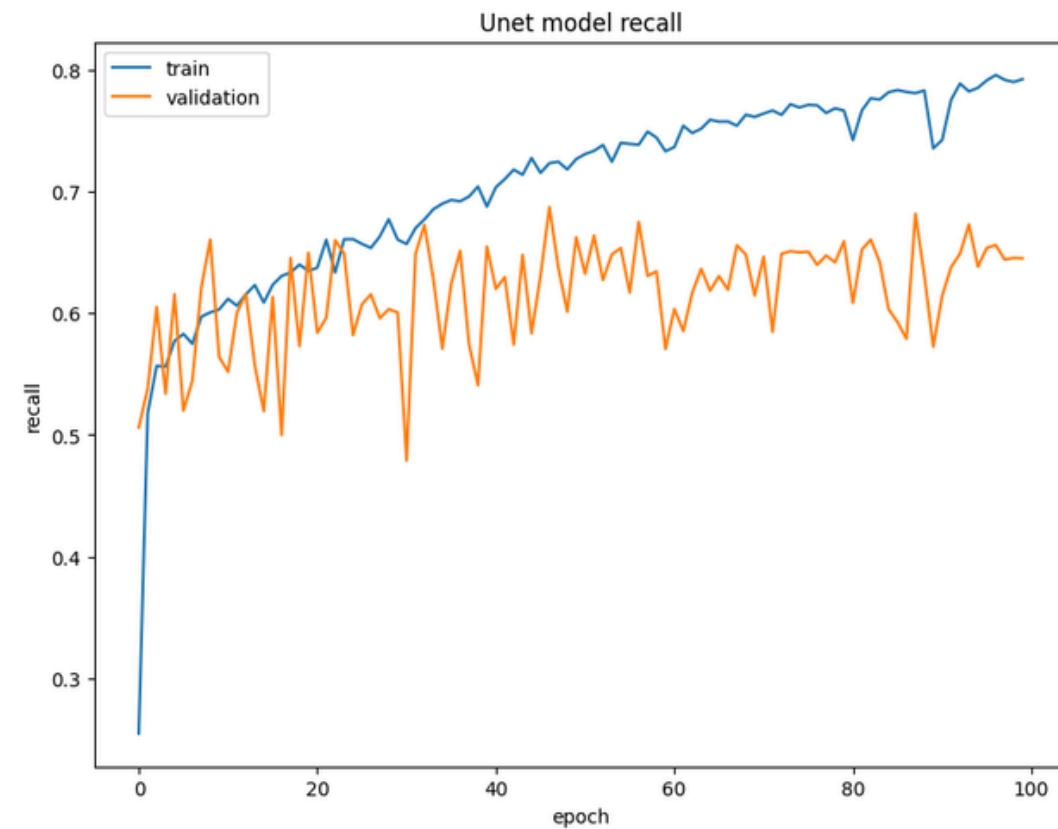
-

EVALUATION METRICS

(PLOTING FROM TRAINING PROGRESS)

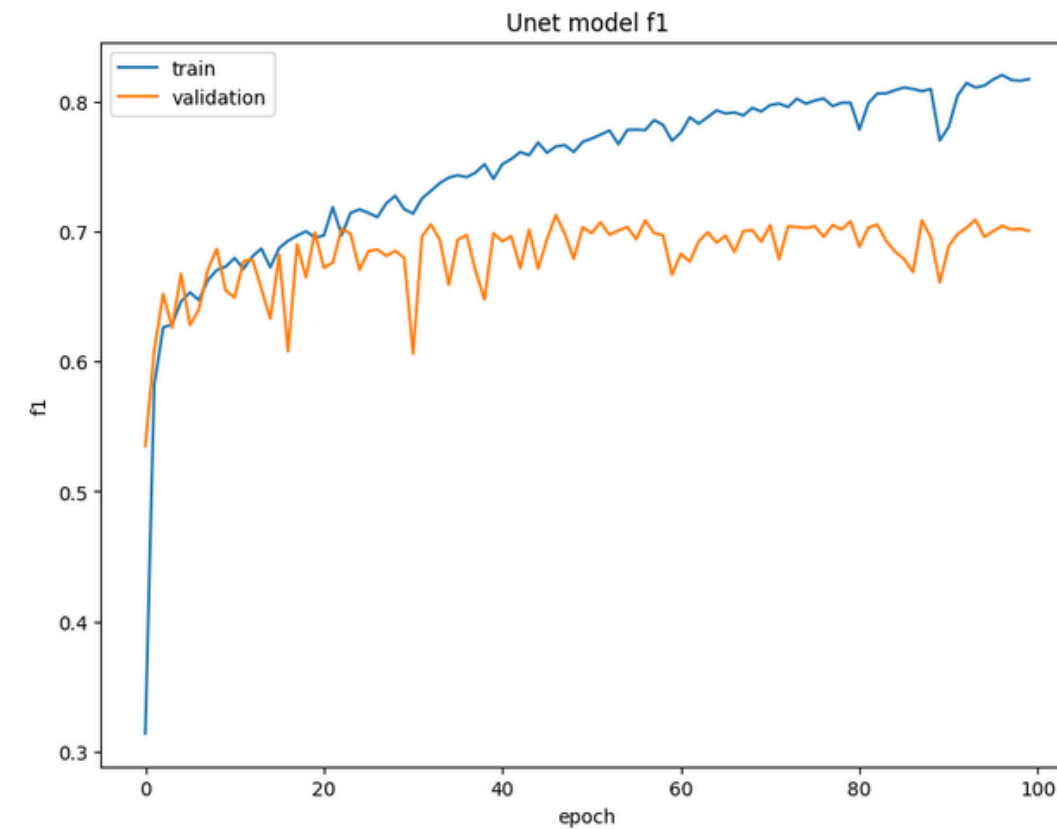


Evaluation Metrics and
Visualization Prediction



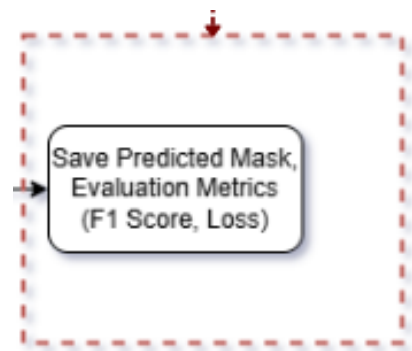
- **Recall:**

- Pro: Initial validation recall is good, meaning the model finds a significant portion of actual landslide areas.
- Con: Validation recall plateaus lower and fluctuates, implying it misses some actual landslides consistently in new images.



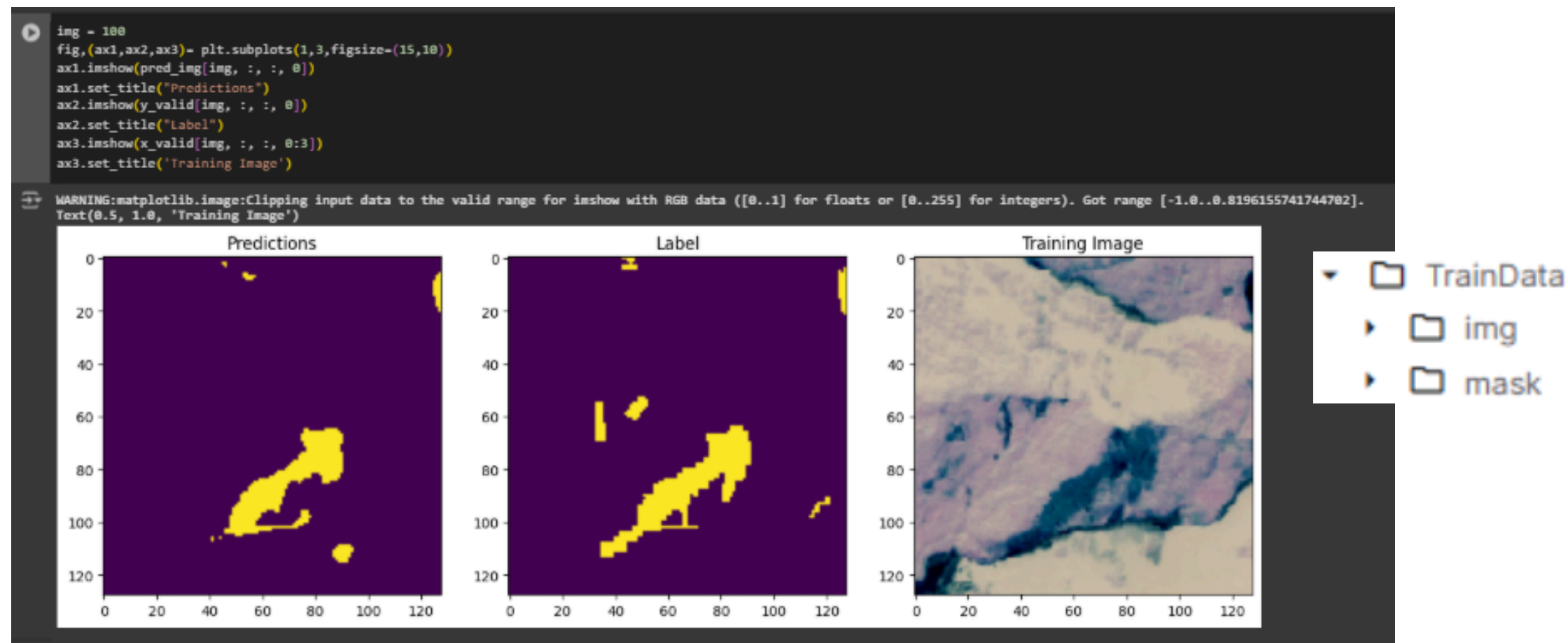
- **F1-Score:**

- Pro: Validation F1-Score quickly reaches a strong overall balance between correctly identified landslides and minimal false alarms.
- Con: It plateaus early and fluctuates, confirming that the model's overall landslide detection performance on new data is limited and inconsistent due to overfitting.



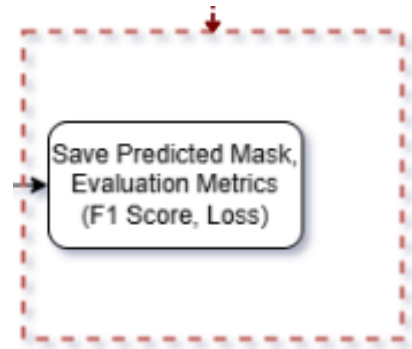
Evaluation Metrics and
Visualization Prediction

PREDICTIONS-LABEL-TRAINING IMAGE (AFTER TRAINING)



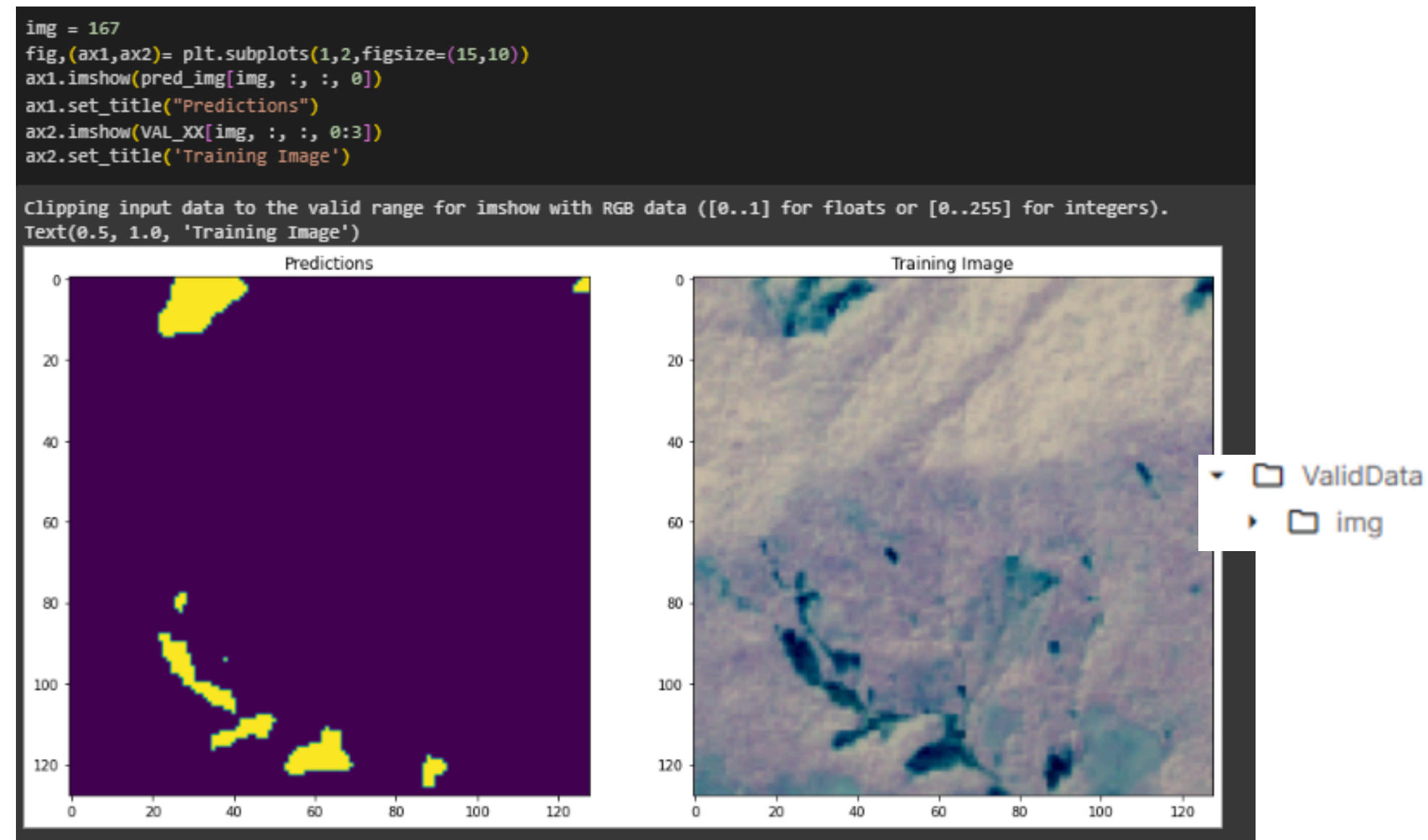
We can take an image that comes from train data, then the model will predict it.

This visualization shows the model's predictions on validation data that **originated from the same initial dataset** as the training data



Evaluation Metrics and
Visualization Prediction

VISUALIZATION OF VALIDATION IMAGE



We can take an image that comes from valid data, then the model will predict it.

So, This visualization displays predictions **on a completely separate**, external validation dataset that the model **had never seen** during any part of the training or internal validation process

FINAL MODEL PERFORMANCE (REVISION ADDED)

KEY METRICS ACHIEVED:

Loss: 0.0382

This low loss value indicates that the model's predictions are very close to the true labels on the validation set, signifying effective learning.

Accuracy: 0.9876 (98.76%)

The remarkably high accuracy suggests that the model correctly classifies a vast majority of pixels (both landslide and non-landslide).

F1-Score: 0.7106

This metric provides a balanced view of precision and recall. An F1-Score of around 0.71 indicates a good overall performance in identifying landslides, especially considering potential class imbalance.

Precision: 0.7859

A high precision means that when the model predicts a pixel as a landslide, it is correct nearly 79% of the time, minimizing false alarms.

Recall: 0.6523

This recall value indicates that the model successfully identifies about 65% of all actual landslide pixels present in the validation data.

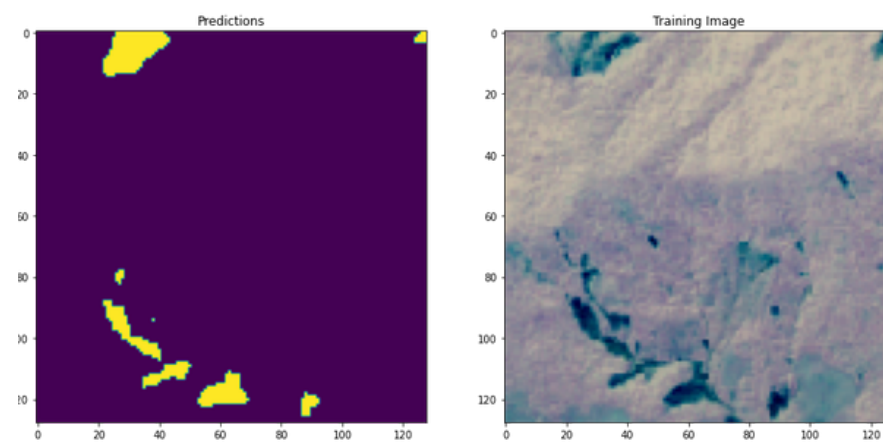
```
[ ] loss, accuracy, f1_score, precision, recall = model.evaluate(x_valid, y_valid, verbose=0)
    print(loss, accuracy, f1_score, precision, recall)

🔄 0.03824269026517868 0.9875717759132385 0.7106499671936035 0.7850735187530518 0.6522870063781738
```

Conclusion Analysis:

The high **accuracy** is a positive sign, but given the imbalanced nature of landslide detection (landslide areas are usually small), the **F1-Score** provides a more reliable assessment. An F1-Score of 0.7106 demonstrates a strong capability in balancing precision and recall. While **precision** is higher (0.7859), suggesting fewer false positives, the **recall** (0.6523) indicates some actual landslide areas might still be missed. This trade-off is common and points to areas for future optimization, possibly by further addressing class imbalance or exploring architectural enhancements.

QUALITATIVE ANALYSIS



The model shows a promising ability to identify potential landslide areas (yellow regions), successfully capturing irregular features from the input image.

However, there are limitations to this analysis from the system:

Due to the very large dataset and the constrained GPU resources on platforms like Google Colab, it is computationally intensive to re-compile and retrain the model efficiently. This limits my ability to robustly and comprehensively assess the model's performance across various complexities within the entire final validation dataset.

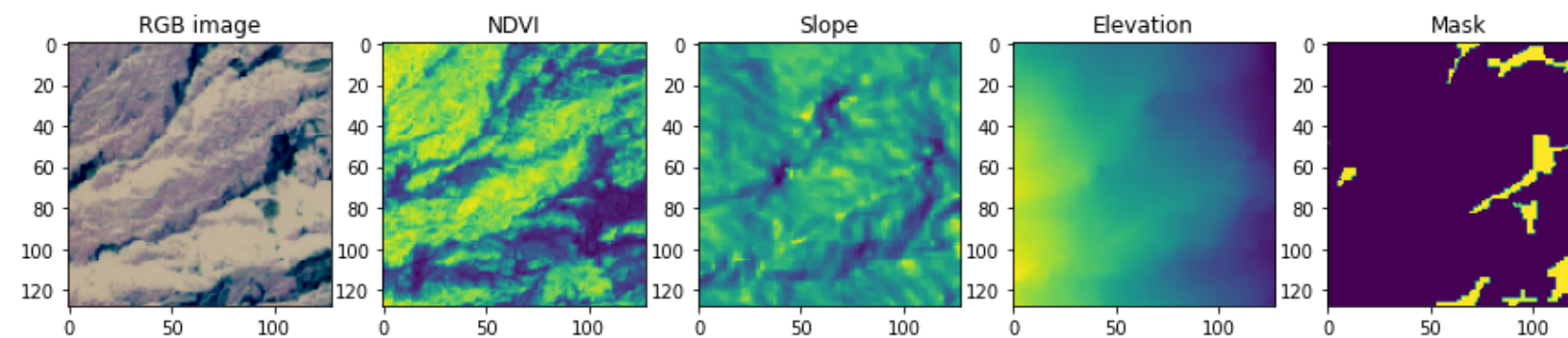
```
1 TrainData/img/image_10.h5 TrainData/mask/mask_10.h5
2 TrainData/img/image_100.h5 TrainData/mask/mask_100.h5
3 TrainData/img/image_1000.h5 TrainData/mask/mask_1000.h5
4 TrainData/img/image_1001.h5 TrainData/mask/mask_1001.h5
5 TrainData/img/image_1002.h5 TrainData/mask/mask_1002.h5
6 TrainData/img/image_1003.h5 TrainData/mask/mask_1003.h5
7 TrainData/img/image_1004.h5 TrainData/mask/mask_1004.h5
8 TrainData/img/image_1005.h5 TrainData/mask/mask_1005.h5
9 TrainData/img/image_1006.h5 TrainData/mask/mask_1006.h5
10 TrainData/img/image_1007.h5 TrainData/mask/mask_1007.h5
11 TrainData/img/image_1008.h5 TrainData/mask/mask_1008.h5
12 TrainData/img/image_1009.h5 TrainData/mask/mask_1009.h5
13 TrainData/img/image_101.h5 TrainData/mask/mask_101.h5
14 TrainData/img/image_1010.h5 TrainData/mask/mask_1010.h5
15 TrainData/img/image_1011.h5 TrainData/mask/mask_1011.h5
16 TrainData/img/image_1012.h5 TrainData/mask/mask_1012.h5
17 TrainData/img/image_1013.h5 TrainData/mask/mask_1013.h5
18 TrainData/img/image_1014.h5 TrainData/mask/mask_1014.h5
19 TrainData/img/image_1015.h5 TrainData/mask/mask_1015.h5
20 TrainData/img/image_1016.h5 TrainData/mask/mask_1016.h5
21 TrainData/img/image_1017.h5 TrainData/mask/mask_1017.h5
22 TrainData/img/image_1018.h5 TrainData/mask/mask_1018.h5
23 TrainData/img/image_1019.h5 TrainData/mask/mask_1019.h5
24 TrainData/img/image_102.h5 TrainData/mask/mask_102.h5
25 TrainData/img/image_1020.h5 TrainData/mask/mask_1020.h5
26 TrainData/img/image_1021.h5 TrainData/mask/mask_1021.h5
27 TrainData/img/image_1022.h5 TrainData/mask/mask_1022.h5
28 TrainData/img/image_1023.h5 TrainData/mask/mask_1023.h5
29 TrainData/img/image_1024.h5 TrainData/mask/mask_1024.h5
30 TrainData/img/image_1025.h5 TrainData/mask/mask_1025.h5
31 TrainData/img/image_1026.h5 TrainData/mask/mask_1026.h5
32 TrainData/img/image_1027.h5 TrainData/mask/mask_1027.h5
33 TrainData/img/image_1028.h5 TrainData/mask/mask_1028.h5
34 TrainData/img/image_1029.h5 TrainData/mask/mask_1029.h5
35 TrainData/img/image_103.h5 TrainData/mask/mask_103.h5
```

In essence, to collect the final validation data, we have to repeat the process from the beginning where in one training process, because it will train thousands of images and also evaluate, which one process will take 20-30 minutes.

CONCLUSION

This project followed a comprehensive deep learning pipeline: environment setup & data testing (1-2), multi-channel data preparation (3), U-Net model definition & training (4), performance evaluation & prediction visualization (5), and final validation on external data with result saving (6).

In conclusion, this project successfully developed a U-Net-based deep learning framework for the semantic segmentation of landslides. The model demonstrated sufficient capability in identifying landslide-affected areas from satellite imagery



Proven Framework and Future Work

The project provides a validated end-to-end deep learning framework that can be adapted for similar semantic segmentation tasks in **geospatial analysis**. In future, we can expanding the training dataset to include a wider variety of geographical regions and landslide types is crucial to improve the model's generalization capabilities. Of course, This would require access to more substantial computational resources.

THANK YOU

Reference :

O. Ghorbanzadeh, Y. Xu, P. Ghamisi, M. Kopp, and D. Kreil, "Landslide4Sense: Reference Benchmark Data and Deep Learning Models for Landslide Detection," Zenodo, 2023. [Online]. Available: <https://zenodo.org/records/10463239>

H. Bhatia, "U-Net Architecture Explained," GeeksforGeeks, May 23, 2024. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/u-net-architecture-explained/>

T. B. Kshetri, "Landslide4Sense," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/tekbahadurkshetri/landslide4sense>

IARAI, "Landslide4Sense-2022." GitHub. [Online]. Available: <https://github.com/iarai/Landslide4Sense-2022>

iamtekson, "landslide4sense-solution." GitHub. [Online]. Available: <https://github.com/iamtekson/landslide4sense-solution>

